

Hedge: Tiny Learned Routing vs. Tuned Fixed Interpolation in Hybrid Language Models — A Controlled Two-Corpus Study

Chiheb Nabil

Remote Skills LTD

chiheb.design@gmail.com

Abstract

Hybrid language models that combine neural networks with n -gram statistics conventionally blend them with a small number of fixed, corpus-level weights (Mikolov et al., 2011; Sundermeyer et al., 2012). We test whether replacing every fixed weight with a *learned per-token router* changes the picture in the small-footprint regime that on-device applications occupy. HEDGE mixes six frozen experts—a Kneser-Ney trigram, two recency bigram experts, a 512-token cache, a continuation unigram, and one small GRU—using an MLP router of 2.6–13.5 KB that reads both the experts’ current votes and cheap context evidence. Under a fixed aligned-scoring protocol on WikiText-2 and the Penn Treebank, the learned router beats a *tuned* fixed interpolation of the *same* experts at every tested seed (90.3 vs. 135.8 and 68.0 vs. 92.7 test perplexity), while static mixing fails badly (171.0 and 101.7). Ablations locate the routing signal in the interaction between votes and evidence, and a cost harness shows the router adds at most 8% latency over fixed interpolation. All code, logs, and the full reproduction pipeline are released.

1 Introduction

Mixture architectures with learned routing are among the oldest ideas in neural computation (Jacobs et al., 1991; Jordan and Jacobs, 1994) and now scale to frontier models (Shazeer et al., 2017). We claim no architectural novelty. Instead, we ask a question the literature has not answered at the *bottom* of the footprint range: when every expert is a frozen classical statistic plus one small recurrent network, does learned per-token routing still beat the tuned fixed interpolation that practitioners actually deploy?

The question matters because the fixed-weight answer is settled at larger scales: interpolating a neural LM with Kneser-Ney smoothing using one

or a few heldout-tuned weights has been standard practice for over a decade (Mikolov et al., 2011; Sundermeyer et al., 2012; Chang et al., 2015). At those scales the neural component dominates, so the blending scheme is a detail. In the small-footprint regime—on-device, offline, CPU-only personalization (Mathur et al., 2023)—the experts are comparably matched, the router is one of the largest *learnable* surface areas, and nobody has checked whether per-token routing changes the conclusion.

We answer it with a controlled experiment. HEDGE routes among six experts: a modified Kneser-Ney trigram (Kneser and Ney, 1995; Chen and Goodman, 1999), lag-2 and lag-3 recency bigrams, a 512-token adaptive cache, a continuation unigram, and a GRU (Mikolov et al., 2010, 2011) of 3.3–8.2M parameters trained on CPU. The router is a one-hidden-layer MLP of 2.6–13.5 KB whose input pairs the experts’ *votes* (their current top predictions and probabilities) with 15 *evidence* features of the context (recency, entropy, cache state). Its hidden size is the only per-corpus hyperparameter, selected on validation data.

Findings. On two corpora (WikiText-2 (Merity et al., 2016) and the Penn Treebank (Marcus et al., 1993; Mikolov et al., 2011)) under one aligned-scoring protocol:

- The learned router beats tuned fixed-weight interpolation of the same experts at *every* tested seed on both corpora: 90.3 vs. 135.8 (WT-2, −33%) and 68.0 vs. 92.7 (PTB, −27%).
- Static per-corpus mixing fails (171.0 and 101.7): the win comes from *per-token* routing, not from extra parameters.
- The routing signal lives in the votes×evidence interaction: gate on votes alone, 201.8; evidence alone, 367.9 (worse

than equal mixing); both, $\sim 90\text{--}100$.

- The router costs at most 8% extra latency over fixed interpolation (3.60 vs. 3.34 ms/token on WT-2; on PTB it is free).

We release the full stack—every expert, the gate, the aligned evaluation protocol, seed audits, and a one-command reproduction per corpus—to make each number checkable.¹

2 Related Work

Interpolated language models. Linear interpolation of probability estimates originates with [Jelinek \(1980\)](#) and was systematized by Kneser-Ney smoothing ([Kneser and Ney, 1995](#)) and its empirical study ([Chen and Goodman, 1999](#)). Our trigram expert follows this line; our interpolation *weights* do not.

Fixed-weight neural/ n -gram hybrids. RNN and LSTM LMs have long been interpolated with Kneser-Ney models using heldout-tuned fixed weights ([Mikolov et al., 2011](#); [Sundermeyer et al., 2012](#); [Chang et al. \(2015\)](#) discriminatively train a handful of context-dependent scaling factors. Our fixed-interpolation baseline is exactly this family, and our question is whether learned per-token routing changes their conclusion at small footprint.

Mixture of experts. Gating networks date to [Jacobs et al. \(1991\)](#) and [Jordan and Jacobs \(1994\)](#); sparse routing at scale to [Shazeer et al. \(2017\)](#). That lineage learns routers for *trainable* experts, usually neural, at footprints orders of magnitude above ours. We route among *frozen* classical experts plus one small network, beneath any footprint those systems report.

Prediction with expert advice. Our hindsight oracle—the per-position best expert—is the classic competitor bound ([Freund and Schapire, 1997](#); [Cesa-Bianchi and Lugosi, 2006](#)). We use it as an honest upper bound on what any router could extract from these experts.

3 The HEDGE System

Experts. Six experts, all frozen after training: (1) a modified Kneser-Ney trigram (discount scale

	WT-2	PTB
train tokens	$\approx 2.0\text{M}$	907,138
valid tokens	214k	71,825
test tokens	245k	80,594
test positions (aligned)	217,004	75,623
vocabulary	33k	9,644

Table 1: Corpora. PTB uses the Mikolov split with vocabulary-from-train and OOV mapped to `unk`; WT-2 follows the standard word-level setup.

1.15, shared across corpora); (2–3) lag-2 and lag-3 recency bigram experts; (4) a 512-token adaptive cache; (5) a continuation unigram; (6) a word-level GRU (vocabulary +1, 3.3M parameters on PTB, 8.2M on WT-2; Adam, cosine schedule, early stopping on validation; batchified perplexity 86.3 on PTB).

Router. A one-hidden-layer MLP with softmax over the six experts. Its 20 input features pair each expert’s *vote* (top prediction, its probability, and rank statistics) with 15 *evidence* features of the current context (cache hit rates, recency densities, token entropy proxies). Hidden size is selected on validation from $\{24, 64, 128, 256\}$; the selected sizes are 13.5 KB (WT-2) and 2.6 KB (PTB). Training is per-position cross-entropy over collected aligned features, with an age-invariant cache-density feature to prevent a train/test distribution leak we detected and fixed (details in the repository dossier).

4 Experimental Protocol

All systems score *one shared gold token stream* per corpus under a fixed aligned protocol: exactly the positions the router sees ($\sum_s \text{len}(s) - 1$ per split), so every row of every table is comparable by construction. Model selection uses validation data only. The fixed-interpolation baseline blends KN3 and the GRU with a single weight λ chosen from a 201-point grid on validation ($\lambda^* = 0.21$ on WT-2, 0.28 on PTB)—it receives the same tuning budget the router’s hidden-size search receives. Headline router rows are audited over three seeds and also reported as 3-router ensembles. The oracle assigns each position its best expert in hindsight; no realizable system can beat it.

5 Results

The router beats tuned fixed interpolation (Table 2). With identical experts and comparable

¹<https://github.com/chihebnabil/hedge>

system	WT-2 test PP	PTB test PP	footprint
pure KN3 trigram	296.2	165.2	21.4 MB
KenLM 5-gram (toolkit defaults)	250.9	138.4	53.8 MB trie
pure GRU (aligned)	146.9	103.0	32.9 MB
5-expert gate (no GRU)	99.7	72.7	21.4 MB + 6.5 KB
fixed-weight KN3+GRU (tuned)	135.8	92.7	54.3 MB
static mixture (6 experts)	171.0	101.7	—
learned router (6 experts)	90.3	68.0	54.3 MB + 2.6–13.5 KB
3-seed mean $\pm$ sd	101.1 $\pm$ 21.2	74.6 $\pm$ 10.1	
3-router ensemble	90.2	69.9	
router trained on valid	81.5 $\pm$ 4.1	70.7 $\pm$ 12.1	
hindsight oracle (bound)	57.4	44.4	—

Table 2: The primary comparison. Word-level test perplexity under one aligned protocol; the router and the fixed-weight baseline use identical experts. The router beats fixed interpolation on both corpora at every seed (worst seeds: 125.6 vs. 135.8; 86.1 vs. 92.7). Batchified GRU evaluation, which scores easy sentence-initial targets, gives 119.2 (WT-2) and 86.3 (PTB); we cite the aligned numbers for comparability.

tuning budgets, learned per-token routing cuts perplexity by 33% (WT-2) and 27% (PTB) relative to the fixed-weight baseline. The dominance is seed-robust: the *worst* router seed still beats the tuned baseline on both corpora, and both ensembles dominate as well. Notably, fixed interpolation prefers a *more* GRU-heavy mixture on PTB ($\lambda^* = 0.28$ vs. 0.21) yet still loses: a single global weight cannot reweight per position.

Static mixing is not enough. A learned *static* mixture of all six experts (per-corpus weights) reaches only 171.0 / 101.7 with near-zero seed variance—the improvement comes from routing *per token*, not from added parameters.

Where the routing signal lives. With the 5-expert stack on WT-2: a gate reading votes only reaches 201.8; evidence only, 367.9 (worse than equal mixing); both, 99.7. The evidence does not say *what* to predict; it says *whose vote to trust*—the signal is the interaction.

Router size is corpus-dependent and validation selects correctly. Sweeping hidden size {24, 64, 128, 256}: WT-2 prefers 13.5 KB (test 144.1/92.2/**90.3**/120.5), PTB prefers 2.6 KB (test **68.0**/87.5/78.8/84.4). Both optima sit far below any neural component; selection on validation picks the right one on both corpora.

Cost of routing. Best-of-3 causal wall-clock over 200 test sentences (batch-1, 4 CPU cores; energy = wall $\times$ 15 W TDP proxy): full system 3.60 ms/token (WT-2) and 1.02 ms/token (PTB) versus 3.34/1.21 for fixed interpolation — the router’s share is $\leq 8\%$. The GRU dominates cost (5.09 ms/token alone on WT-2); total footprint is

68.7 MB (WT-2) / 28.7 MB (PTB).

6 Limitations

We report these explicitly. (1) *Scale*: WT-2 and PTB are small corpora; absolute numbers are far from modern LSTM recipes (Zaremba et al., 2014; Merity et al., 2018)—our claim is the controlled comparison, not state of the art. (2) *The wrinkle*: on PTB, the larger 13.5 KB router overfits (78.8, worse than the 5-expert gate’s 72.7); the valid-selected 2.6 KB router and the valid-trained router do not. Router variance on the smaller corpus is real (sd 10.1) and we report it. (3) *Baselines*: we include a toolkit-grade KenLM 5-gram (Heafield, 2011) trained with default settings on the same streams (250.9 and 138.4 aligned test PP): it improves on our trigram expert, as expected of a battle-tested implementation, but sits far above every hybrid row. We do not compare against transformer LMs—the natural stronger baseline, left to the extension. (4) One tokenizer, two English corpora, CPU timing on one machine—all documented in the released dossier.

7 Conclusion and Future Work

In the small-footprint regime, a router of 2.6–13.5 KB extracts 27–33% perplexity reduction from *the same experts* that tuned fixed interpolation blends—and static mixing cannot. We take this as evidence that per-token routing, not parameter count, is where the small hybrid’s headroom lives. The immediate extensions of this workshop study are a third larger corpus (WikiText-103) and a tiny transformer expert; an extended version incorporating them is in preparation and will cite this one.

References

- Nicolò Cesa-Bianchi and Gábor Lugosi. 2006. *Prediction, Learning, and Games*. Cambridge University Press.
- Shuo Chang, Anmol Lahiri, Isaac Alphonso, Barlas Oguz, and Mikhail Levit. 2015. [Discriminative training of context-dependent language model scaling factors and interpolation weights](#). In *IEEE Workshop on Automatic Speech Recognition and Understanding (ASRU)*.
- Stanley F. Chen and Joshua Goodman. 1999. An empirical study of smoothing methods for language models. *Computational Linguistics*, 25(1):73–100.
- Yoav Freund and Robert E. Schapire. 1997. [A decision-theoretic generalization of on-line learning and an application to boosting](#). *Journal of Computer and System Sciences*, 55(1):119–139.
- Kenneth Heafield. 2011. KenLM: Faster and smaller language model queries. In *Proceedings of the Sixth Workshop on Statistical Machine Translation (WMT)*.
- Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. 1991. [Adaptive mixtures of local experts](#). *Neural Computation*, 3(1):79–87.
- Frederick Jelinek. 1980. Interpolated estimation of Markov source parameters from sparse data. In Edvard S. Gelsema and Laveen N. Kanal, editors, *Pattern Recognition in Practice*, pages 381–397. North-Holland, Amsterdam.
- Michael I. Jordan and Robert A. Jacobs. 1994. [Hierarchical mixtures of experts and the EM algorithm](#). *Neural Computation*, 6(2):181–214.
- Reinhard Kneser and Hermann Ney. 1995. [Improved backing-off for M-gram language modeling](#). In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*.
- Mitchell P. Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. 1993. Building a large annotated corpus of English: The Penn treebank. *Computational Linguistics*, 19(1):27–33.
- Puneet Mathur, Zhe Liu, Ke Li, Yingyi Ma, and Gil Keren. 2023. [PersonaLM: Language model personalization via domain-distributed span aggregated K-nearest neighbor retrieval](#). In *Findings of the Association for Computational Linguistics: EMNLP*.
- Stephen Merity, Nitish Shirish Keskar, and Richard Socher. 2018. Regularizing and optimizing LSTM language models. In *International Conference on Learning Representations (ICLR)*. ArXiv:1708.02182.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer sentinel mixture models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Tomas Mikolov, Anoop Deoras, Daniel Povey, Lukáš Burget, and Jan Černocký. 2011. [Strategies for training large scale neural network language models](#). In *IEEE Workshop on Automatic Speech Recognition and Understanding (ASRU)*.
- Tomas Mikolov, Martin Karafiát, Lukáš Burget, Jan Černocký, and Sanjeev Khudanpur. 2010. Recurrent neural network based language model. In *Inter-speech*.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*. ArXiv:1701.06538.
- Martin Sundermeyer, Ralf Schlüter, and Hermann Ney. 2012. [LSTM neural networks for language modeling](#). In *Interspeech*.
- Wojciech Zaremba, Ilya Sutskever, and Oriol Vinyals. 2014. Recurrent neural network regularization. *arXiv preprint arXiv:1409.2329*.

A Reproduction

Every number above regenerates from the released repository with one command per stage per corpus (data → gold streams → trigram → GRU → gate features → main results table → seed audit), with a corpus environment variable switching all artifact paths. Negative results (a decorative “novel” prior in the base repository; a 5-gram baseline artifact we correct from 302.5 to 294.2, still ≈ the trigram) are documented and reproducible.